- MCP Tools Architecture and Flow
  - Table of Contents
  - Overview
    - Key Benefits
  - Architecture Diagram
  - Component Breakdown
    - 1. Tool Definition (tools/*.py)
      - Tool Descriptor
      - Handler Function
    - 2. Tool Registry (tools/init.py)
    - 3. MCP Server (server.py)
    - 4. MCP Client (client.py)
    - 5. LangGraph Integration (graph_v2.py)
  - Tool Definition Pattern
    - Standard Tool Structure
    - Best Practices
  - Tool Execution Flow
    - Complete Request Lifecycle
    - Tool Call Example
  - Tool Registry System
    - Registry Structure
    - Registry Benefits
    - Adding a New Tool
  - MCP Client-Server Communication
    - Connection Setup
    - Tool Discovery
    - Tool Invocation
    - Transport Layer
  - Integration with LangGraph
    - Tool Binding to Agents
    - Tool Execution Node
    - Conditional Routing
    - Tool Loop Pattern
  - Tool Categories
    - 1. Database Tools (db_tools.py)
    - 2. Python Execution Tools (python_executor.py)
    - 3. Report Tools (report_tools.py)

- Data Flow Examples
  - Example 1: Loading and Analyzing Data
  - Example 2: Multi-Mill Comparison
- Error Handling
  - Tool-Level Errors
  - Node-Level Errors
  - Agent-Level Recovery
- Extension Guide
  - Adding a New Tool: Step-by-Step
    - Step 1: Create Tool File
    - Step 2: Register Tool
    - Step 3: Assign to Agents (Optional)
    - Step 4: Test
  - Tool Design Guidelines
  - Common Patterns
    - Pattern 1: Data Loading Tool
    - Pattern 2: Computation Tool
    - Pattern 3: File I/O Tool
- Summary
  - Key Takeaways
  - Next Steps

MCP Tools Architecture and Flow

Table of Contents

- Overview
- Architecture Diagram
- Component Breakdown
- Tool Definition Pattern
- Tool Execution Flow
- Tool Registry System
- MCP Client-Server Communication
- Integration with LangGraph
- Tool Categories
- Data Flow Examples

- Error Handling
- Extension Guide

Overview

The agentic system uses **MCP (Model Context Protocol)** to provide tools to LangGraph agents. This architecture separates tool implementation from agent logic, enabling:

- **Modular tool development** - Tools are independent, testable units
- **Dynamic tool discovery** - Agents discover available tools at runtime
- **Protocol-based communication** - Standardized client-server interface
- **Multi-agent coordination** - Multiple agents share the same tool set

Key Benefits

1. **Separation of Concerns**: Tools focus on data operations, agents focus on reasoning
2. **Reusability**: Same tools can be used across different agent workflows
3. **Testability**: Tools can be tested independently of the graph
4. **Scalability**: New tools can be added without modifying agent code

---

Architecture Diagram

```
┌──────────────────────────────────────────────────────────────────────────┐
│                              USER REQUEST                                 │
│                          "Analyze Mill 8 for 30 days"                     │
└──────────────────────────────────────────────────────────────────────────┘
                                   │
                                   │
                                   ▼
┌──────────────────────────────────────────────────────────────────────────┐
│                              main.py                                      │
│   ┌──────────────────────────────────────────────────────────────────┐   │
│   │  1. Load API key from .env                                        │   │
│   │  2. Connect to MCP server (localhost:8003/mcp)                    │   │
│   │  3. Fetch tools via client.get_mcp_tools()                        │   │
│   │  4. Build graph with build_graph(tools, api_key)                  │   │
│   │  5. Invoke graph with user question                               │   │
```

```
client.py

get_mcp_tools(session)
  ├─ session.list_tools() → Get tool descriptors
  └─ mcp_tool_to_langchain() → Wrap each tool
      ├─ _json_schema_to_pydantic() → Create args_schema
      ├─ StructuredTool.from_function() → LangChain wrapper
      └─ Returns list[BaseTool] for LangGraph
```

```
MCP Server (server.py)

Server("agentic-data-analysis-server")
  ├─ @server.list_tools() → Returns all tool descriptors
  └─ @server.call_tool() → Dispatches to handler

StreamableHTTPSessionManager → Handles HTTP transport
Starlette app → Mounts at /mcp endpoint
```

```
Tool Registry (tools/__init__.py)

tools = {
  "get_db_schema": {
    "tool": get_db_schema_tool,      ← Tool descriptor
    "handler": get_db_schema           ← Handler function
  },
  "query_mill_data": { ... },
  "execute_python": { ... },
  ...
}
```

```
Tool Implementations (tools/)

db_tools.py                 python_executor.py
├─ get_db_schema            ├─ execute_python
├─ query_mill_data
└─ query_combined_data

report_tools.py
├─ list_output_files
└─ write_markdown_report
```

```
│   └──────────────────────┴──────────────────────────────────────────────┘   │
└──────────────────────────────────────────────────────────────────────────────┘
```

Component Breakdown

1. Tool Definition (tools/*.py)

Each tool consists of two parts:

Tool Descriptor

```
query_mill_data_tool = types.Tool(
    name="query_mill_data",
    description="Load time-series process data for a specific mill...",
    inputSchema={
        "type": "object",
        "properties": {
            "mill_number": {"type": "integer", "description": "..."},
            "start_date": {"type": "string", "description": "..."},
            # ...
        },
        "required": ["mill_number"],
    },
)
```

Handler Function

```
async def query_mill_data(arguments: dict) -> list[types.TextContent]:
    mill_number = arguments.get("mill_number")
    # ... implementation ...
    return [types.TextContent(type="text", text=json.dumps(result))]
```

2. Tool Registry (tools/init.py)

Central registry mapping tool names to their implementations:

```
tools = {
    "query_mill_data": {
```

```
        "tool": query_mill_data_tool,      # Descriptor
        "handler": query_mill_data          # Handler
    },
    # ... other tools
}
```

Purpose: Single source of truth for available tools. Adding a new tool requires only:

1. Creating the descriptor and handler
2. Adding one entry to the registry

3. MCP Server (server.py)

Low-level MCP server that exposes tools via HTTP:

```
server = Server("agentic-data-analysis-server", lifespan=server_lifespan)

@server.list_tools()
async def handle_list_tools() -> list[types.Tool]:
    return [entry["tool"] for entry in tools.values()]

@server.call_tool()
async def handle_call_tool(name: str, arguments: dict) -> list[types.TextContent]:
    if name not in tools:
        raise ValueError(f"Unknown tool: {name}")
    handler = tools[name]["handler"]
    return await handler(arguments)
```

Key Features:

- **Lifespan management**: Startup/shutdown hooks
- **Tool discovery**: `list_tools()` returns all available tools
- **Tool dispatch**: `call_tool()` routes to correct handler
- **HTTP transport**: StreamableHTTPSessionManager for web access

4. MCP Client (client.py)

Bridges MCP tools to LangChain's tool system:

```
async def get_mcp_tools(session: ClientSession) -> list[BaseTool]:
    tools_result = await session.list_tools()
    langchain_tools = [mcp_tool_to_langchain(t, session) for t in
```

```
tools_result.tools]
    return langchain_tools

def mcp_tool_to_langchain(tool: mcp_types.Tool, session: ClientSession) ->
BaseTool:
    args_schema = _json_schema_to_pydantic(tool.inputSchema, model_name=tool.name)

    async def _call(**kwargs: Any) -> str:
        result = await session.call_tool(name=tool.name, arguments=kwargs)
        return result.content[0].text

    return StructuredTool.from_function(
        coroutine=_call,
        name=tool.name,
        description=tool.description,
        args_schema=args_schema,
    )
```

Key Features:

- **Schema conversion**: JSON Schema → Pydantic model for LangChain
- **Async wrapper**: Converts MCP tool calls to LangChain-compatible functions
- **Session capture**: Closes over session for persistent connection

5. LangGraph Integration (graph_v2.py)

Tools are bound to specialist LLMs:

```
tools_by_name = {t.name: t for t in tools}
TOOL_SETS = {
    "data_loader": ["query_mill_data", "query_combined_data", "get_db_schema"],
    "analyst":     ["execute_python", "list_output_files"],
    "code_reviewer": ["execute_python", "list_output_files"],
    "reporter":    ["list_output_files", "write_markdown_report"],
}

specialist_llms = {}
for stage_name, tool_names in TOOL_SETS.items():
    stage_tools = [tools_by_name[n] for n in tool_names if n in tools_by_name]
    specialist_llms[stage_name] = llm.bind_tools(stage_tools)
```

Tool Execution:

```
async def tool_node(state: AnalysisState) -> dict:
    last_message = state["messages"][-1]
    results = []
    for tc in last_message.tool_calls:
```

```
        tool = tools_by_name.get(tc["name"])
        output = await tool.ainvoke(tc["args"])
        results.append(ToolMessage(content=str(output), ...))
    return {"messages": results}
```

Tool Definition Pattern

Standard Tool Structure

```
# 1. Input Schema (JSON Schema format)
tool_input_schema = {
    "type": "object",
    "properties": {
        "param1": {
            "type": "string",
            "description": "Description of parameter 1"
        },
        "param2": {
            "type": "integer",
            "description": "Description of parameter 2"
        },
    },
    "required": ["param1"],  # Required parameters
}

# 2. Tool Descriptor
my_tool = types.Tool(
    name="my_tool",
    description="Clear, concise description of what the tool does",
    inputSchema=tool_input_schema,
)

# 3. Handler Function
async def my_tool_handler(arguments: dict) -> list[types.TextContent]:
    # Extract parameters
    param1 = arguments.get("param1")
    param2 = arguments.get("param2")

    # Implementation logic
    result = {"status": "success", "data": "..."}

    # Return as TextContent
    return [types.TextContent(type="text", text=json.dumps(result))]
```

Best Practices

1. **Descriptive Names**: Use clear, action-oriented names (e.g., `query_mill_data` not `get_data`)
2. **Detailed Descriptions**: Help LLMs understand when to use the tool
3. **Type Safety**: Use proper JSON Schema types
4. **Error Handling**: Return error messages in structured format
5. **Compact Output**: Truncate large outputs to stay within token limits

---

Tool Execution Flow

Complete Request Lifecycle

```
┌──────────────────────────────────────────────────────────────────────────┐
│ 1. User Request                                                          │
│    "Analyze Mill 8 for the last 30 days"                                 │
└──────────────────────────────────────────────────────────────────────────┘
                                  │
                                  │
                                  ▼
┌──────────────────────────────────────────────────────────────────────────┐
│ 2. Graph Invocation (main.py)                                            │
│    graph.ainvoke({"messages": [HumanMessage(content=user_input)]})       │
└──────────────────────────────────────────────────────────────────────────┘
                                  │
                                  │
                                  ▼
┌──────────────────────────────────────────────────────────────────────────┐
│ 3. Agent Processing (graph_v2.py)                                        │
│    data_loader agent receives user request                              │
│    └─ LLM decides to call query_mill_data tool                          │
└──────────────────────────────────────────────────────────────────────────┘
                                  │
                                  │
                                  ▼
┌──────────────────────────────────────────────────────────────────────────┐
│ 4. Tool Call (tool_node)                                                 │
│    Extracts tool_calls from last AIMessage                              │
│    └─ Calls tool.ainvoke(arguments)                                     │
└──────────────────────────────────────────────────────────────────────────┘
                                  │
                                  │
                                  ▼
┌──────────────────────────────────────────────────────────────────────────┐
│ 5. LangChain Tool Wrapper (client.py)                                    │
│    StructuredTool._call(**kwargs)                                        │
│    └─ session.call_tool(name=tool.name, arguments=kwargs)               │
└──────────────────────────────────────────────────────────────────────────┘
                                  │
                                  │
                                  ▼
┌──────────────────────────────────────────────────────────────────────────┐
│ 6. MCP Server Dispatch (server.py)                                       │
│    handle_call_tool(name, arguments)                                    │
```

```
│     └─ handler = tools[name]["handler"]                                  │
│     └─ return await handler(arguments)                                   │
└──────────────────────────────────┬──────────────────────────────────────┘
                                   │
                                   ▼

┌──────────────────────────────────────────────────────────────────────────┐
│ 7. Tool Handler Execution (tools/db_tools.py)                            │
│    async def query_mill_data(arguments):                                 │
│     └─ Connect to database                                               │
│     └─ Execute query                                                     │
│     └─ Store DataFrame in memory                                         │
│     └─ Return result as TextContent                                      │
└──────────────────────────────────┬──────────────────────────────────────┘
                                   │
                                   ▼

┌──────────────────────────────────────────────────────────────────────────┐
│ 8. Result Propagation                                                     |
│    ToolMessage(content=tool_result) → Added to state.messages            │
│     └─ Agent receives result                                             │
│     └─ Continues reasoning or calls more tools                           │
└──────────────────────────────────────────────────────────────────────────┘
```

Tool Call Example

Agent Decision:

```
[AIMessage]
I need to load data for Mill 8. I'll call query_mill_data with:
- mill_number: 8
- start_date: "2025-02-12"
- end_date: "2025-03-13"

tool_calls: [
  {
    "name": "query_mill_data",
    "arguments": {"mill_number": 8, "start_date": "2025-02-12", "end_date": "2025-
03-13"}
  }
]
```

Tool Execution:

```
# tool_node extracts tool_calls
for tc in last_message.tool_calls:
    tool = tools_by_name.get(tc["name"])  # query_mill_data
    output = await tool.ainvoke(tc["args"])
    # → StructuredTool._call() → session.call_tool() → handler()
```

Handler Execution:

```python
async def query_mill_data(arguments: dict):
    mill_number = arguments["mill_number"]  # 8
    start_date = arguments["start_date"]    # "2025-02-12"
    end_date = arguments["end_date"]        # "2025-03-13"

    # Database query
    df = pd.read_sql_query(f"SELECT * FROM mills.MILL_08 WHERE ...")

    # Store in memory
    set_dataframe(df, "mill_data_8")

    # Return result
    return [types.TextContent(type="text", text=json.dumps({
        "status": "loaded",
        "rows": len(df),
        "date_range": {"start": "...", "end": "..."}
    }))]
```

Result Propagation:

```
[ToolMessage]
name: query_mill_data
content: {"status": "loaded", "rows": 43200, "date_range": {...}}
```

Tool Registry System

Registry Structure

```python
# tools/__init__.py
tools = {
    "tool_name": {
        "tool": ToolDescriptor,      # MCP Tool object (name, description, schema)
        "handler": Callable,          # Async function that executes the tool
    }
}
```

Registry Benefits

1. **Centralized Discovery**: All tools in one place
2. **Dynamic Loading**: Server reads registry at runtime
3. **Easy Extension**: Add new tools without modifying server code
4. **Type Safety**: Registry enforces tool structure

Adding a New Tool

```
# 1. Create descriptor and handler in appropriate file
my_new_tool = types.Tool(
    name="my_new_tool",
    description="Does something useful",
    inputSchema={...},
)

async def my_new_tool_handler(arguments: dict) -> list[types.TextContent]:
    # Implementation
    return [types.TextContent(type="text", text=json.dumps(result))]

# 2. Add to registry
tools = {
    # ... existing tools
    my_new_tool.name: {
        "tool": my_new_tool,
        "handler": my_new_tool_handler,
    },
}
```

No other changes needed! Server and client automatically discover the new tool.

MCP Client-Server Communication

Connection Setup

```
# main.py
async with streamable_http_client(SERVER_URL) as (read, write, _):
    async with ClientSession(read, write) as session:
        await session.initialize()

        # Session is now ready
        langchain_tools = await get_mcp_tools(session)
```

Tool Discovery

```python
# client.py
async def get_mcp_tools(session: ClientSession) -> list[BaseTool]:
    # 1. List all available tools
    tools_result = await session.list_tools()

    # 2. Convert each MCP tool to LangChain tool
    langchain_tools = [
        mcp_tool_to_langchain(tool, session)
        for tool in tools_result.tools
    ]

    return langchain_tools
```

Tool Invocation

```python
# Inside StructuredTool._call()
async def _call(**kwargs: Any) -> str:
    # 1. Call MCP tool via session
    result = await session.call_tool(
        name=tool.name,
        arguments=kwargs
    )

    # 2. Handle errors
    if result.isError:
        return f"Error: {result.content[0].text}"

    # 3. Return result text
    return result.content[0].text
```

Transport Layer

StreamableHTTPSessionManager provides:

- **HTTP streaming**: Efficient bidirectional communication
- **Connection pooling**: Reuses connections for multiple calls
- **Error handling**: Automatic retries and timeout management

Starlette app exposes:

- **Endpoint**: `/mcp` for MCP protocol

- **Middleware**: Handles MCP-specific headers and framing
- **Async support**: Non-blocking I/O for concurrent requests

---

Integration with LangGraph

Tool Binding to Agents

```
# graph_v2.py
# Each specialist gets specific tools
TOOL_SETS = {
    "data_loader": ["query_mill_data", "query_combined_data", "get_db_schema"],
    "analyst":     ["execute_python", "list_output_files"],
    "code_reviewer": ["execute_python", "list_output_files"],
    "reporter":    ["list_output_files", "write_markdown_report"],
}

# Bind tools to LLMs
for stage_name, tool_names in TOOL_SETS.items():
    stage_tools = [tools_by_name[n] for n in tool_names]
    specialist_llms[stage_name] = llm.bind_tools(stage_tools)
```

Tool Execution Node

```
async def tool_node(state: AnalysisState) -> dict:
    last_message = state["messages"][-1]
    results = []

    for tc in last_message.tool_calls:
        tool = tools_by_name.get(tc["name"])
        if tool:
            try:
                output = await tool.ainvoke(tc["args"])
                results.append(ToolMessage(
                    content=str(output),
                    tool_call_id=tc["id"],
                    name=tc["name"],
                ))
            except Exception as e:
                results.append(ToolMessage(
                    content=f"Error: {e}",
                    tool_call_id=tc["id"],
                    name=tc["name"],
                ))
```

```
    return {"messages": results}
```

Conditional Routing

```
def specialist_router(state: AnalysisState) -> str:
    """After specialist: go to tools if tool_calls, else to manager review."""
    last = state["messages"][-1]
    if hasattr(last, "tool_calls") and last.tool_calls:
        return "tools"
    return "manager_review"
```

Tool Loop Pattern

```
specialist_node → (has tool_calls?) → YES → tool_node → specialist_node
                                              ↓
                                            NO → manager_review_node
```

The loop continues until the specialist finishes (no more tool calls).

Tool Categories

1. Database Tools (db_tools.py)

Tool	Purpose	Inputs	Outputs
`get_db_schema`	Inspect database structure	schema_name	Table/column metadata
`query_mill_data`	Load mill sensor data	mill_number, dates	DataFrame in memory
`query_combined_data`	Load mill + ore quality data	mill_number, dates	Joined DataFrame

Data Storage:

- DataFrames stored in `_dataframes` dict (in-memory)
- Accessible via `get_df(name)` and `list_dfs()`
- Persists across tool calls within a session

2. Python Execution Tools (python_executor.py)

Tool	Purpose	Inputs	Outputs
`execute_python`	Run analysis code	code string	stdout, saved files, errors

Namespace Includes:

- `df`: Default loaded DataFrame
- `get_df(name)`: Get named DataFrame
- `list_dfs()`: List all DataFrames
- `pd, np, plt, sns, scipy.stats`: Analysis libraries
- `OUTPUT_DIR`: Path for saving charts

Security Note: Production should sandbox execution.

3. Report Tools (report_tools.py)

Tool	Purpose	Inputs	Outputs
`list_output_files`	List generated files	extension_filter	File list with sizes
`write_markdown_report`	Write report	filename, content	Saved report file

Output Directory: `agentic/output/` - contains all charts and reports.

Data Flow Examples

Example 1: Loading and Analyzing Data

```
┌──────────────────────────────────────────────────────────────────────────┐
│ STEP 1: Data Loader Agent                                                │
│                                                                          │
│ User: "Analyze Mill 8 for the last 30 days"                              │
│    ↓                                                                     │
│ Agent: Calls get_db_schema() → Sees available tables                     │
│    ↓                                                                     │
│ Agent: Calls query_mill_data(mill_number=8, start_date="2025-02-12", ...)  │
│    ↓                                                                     │
│ Tool: Queries PostgreSQL, stores as "mill_data_8"                        │
│    ↓                                                                     │
│ Result: {"status": "loaded", "rows": 43200, "date_range": {...}}         │
└──────────────────────────────────────────────────────────────────────────┘
                                    │
                                    ▼
┌──────────────────────────────────────────────────────────────────────────┐
│ STEP 2: Analyst Agent                                                    │
│                                                                          │
│ Agent: Calls list_dfs() → Sees "mill_data_8" available                   │
│    ↓                                                                     │
│ Agent: Calls execute_python(code="""                                     │
│     import pandas as pd                                                  │
│     import matplotlib.pyplot as plt                                      │
│     df = get_df('mill_data_8')                                           │
│     print(df[['Ore', 'PSI80']].describe())                               │
│     df[['Ore', 'PSI80']].plot(figsize=(12,6))                            │
│     plt.savefig(os.path.join(OUTPUT_DIR, 'ore_psi80_trend.png'))         │
│ """)                                                                     │
│    ↓                                                                     │
│ Tool: Executes code in namespace with df, plt, OUTPUT_DIR                │
│    ↓                                                                     │
│ Result: {"stdout": "Ore mean: 150.2, PSI80 mean: 23.5...",               │
│          "new_files": ["ore_psi80_trend.png"]}                           │
└──────────────────────────────────────────────────────────────────────────┘
                                    │
                                    ▼
┌──────────────────────────────────────────────────────────────────────────┐
│ STEP 3: Reporter Agent                                                   │
│                                                                          │
│ Agent: Calls list_output_files(extension_filter="png")                   │
│    ↓                                                                     │
│ Result: {"files": [{"name": "ore_psi80_trend.png", "size_kb": 45.2}]}    │
│    ↓                                                                     │
│ Agent: Calls write_markdown_report(filename="mill_8_analysis.md",        │
│                                    content="""...""")                    │
│    ↓                                                                     │
│ Result: {"status": "written", "file": "mill_8_analysis.md", ...}         │
└──────────────────────────────────────────────────────────────────────────┘
```

Example 2: Multi-Mill Comparison

```
┌──────────────────────────────────────────────────────────────────────────┐
│ STEP 1: Data Loader Agent                                                │
│                                                                          │
│ User: "Compare ore loading across all mills for last 72 hours"           │
│    ↓                                                                     │
│ Agent: Loops through mills 1-12, calls query_mill_data for each          │
│    ↓                                                                     │
│ Tools: Stores as "mill_data_1", "mill_data_2", ..., "mill_data_12"       │
└──────────────────────────────────────────────────────────────────────────┘
                                  │
                                  ▼
┌──────────────────────────────────────────────────────────────────────────┐
│ STEP 2: Analyst Agent                                                    │
│                                                                          │
│ Agent: Calls execute_python(code="""                                     │
│     mill_stats = {}                                                      │
│     for i in range(1, 13):                                               │
│         df = get_df(f'mill_data_{i}')                                    │
│         mill_stats[i] = df['Ore'].mean()                                 │
│     print(mill_stats)                                                    │
│     plt.bar(range(1,13), mill_stats.values())                            │
│     plt.savefig(os.path.join(OUTPUT_DIR, 'mill_comparison.png'))         │
│ """)                                                                     │
│    ↓                                                                     │
│ Result: {"stdout": "{1: 145.2, 2: 152.3, ...}",                          │
│          "new_files": ["mill_comparison.png"]}                           │
└──────────────────────────────────────────────────────────────────────────┘
```

Error Handling

Tool-Level Errors

```python
async def query_mill_data(arguments: dict) -> list[types.TextContent]:
    mill_number = arguments.get("mill_number")

    # Validation
    if mill_number is None or not (1 <= mill_number <= 12):
        raise ValueError("mill_number is required and must be between 1 and 12")

    try:
        df = pd.read_sql_query(query, engine)
    except Exception as e:
        return [types.TextContent(type="text", text=json.dumps({
            "status": "error",
            "message": str(e)
        }))]
```

Node-Level Errors

```
async def tool_node(state: AnalysisState) -> dict:
    results = []
    for tc in last_message.tool_calls:
        tool = tools_by_name.get(tc["name"])
        try:
            output = await tool.ainvoke(tc["args"])
            results.append(ToolMessage(content=str(output), ...))
        except Exception as e:
            results.append(ToolMessage(
                content=f"Error: {e}",
                tool_call_id=tc["id"],
                name=tc["name"],
            ))
    return {"messages": results}
```

Agent-Level Recovery

Agents receive error messages and can:

1. Retry with corrected parameters
2. Try alternative tools
3. Report the issue to the user

Extension Guide

Adding a New Tool: Step-by-Step

Step 1: Create Tool File

```
# tools/my_new_tool.py
import json
from mcp import types

# Input schema
my_tool_input_schema = {
    "type": "object",
    "properties": {
        "param1": {"type": "string", "description": "..."},
```

```
    },
    "required": ["param1"],
}

# Tool descriptor
my_tool = types.Tool(
    name="my_tool",
    description="Does something useful",
    inputSchema=my_tool_input_schema,
)

# Handler
async def my_tool_handler(arguments: dict) -> list[types.TextContent]:
    param1 = arguments.get("param1")

    # Implementation
    result = {"status": "success", "data": "..."}

    return [types.TextContent(type="text", text=json.dumps(result))]
```

Step 2: Register Tool

```
# tools/__init__.py
from tools.my_new_tool import my_tool, my_tool_handler

tools = {
    # ... existing tools
    my_tool.name: {
        "tool": my_tool,
        "handler": my_tool_handler,
    },
}
```

Step 3: Assign to Agents (Optional)

```
# graph_v2.py
TOOL_SETS = {
    "data_loader": ["query_mill_data", ..., "my_tool"],  # Add here
    # ...
}
```

Step 4: Test

```
# Test the tool directly
async with streamable_http_client(SERVER_URL) as (read, write, _):
    async with ClientSession(read, write) as session:
```

```
        await session.initialize()
        result = await session.call_tool(
            name="my_tool",
            arguments={"param1": "test"}
        )
        print(result.content[0].text)
```

Tool Design Guidelines

1. **Single Responsibility**: Each tool should do one thing well
2. **Atomic Operations**: Tools should be idempotent when possible
3. **Clear Descriptions**: Help LLMs understand when to use the tool
4. **Structured Output**: Return JSON for easy parsing
5. **Error Messages**: Provide actionable error information

Common Patterns

Pattern 1: Data Loading Tool

```
async def load_data_tool(arguments: dict):
    # 1. Validate inputs
    # 2. Query database/file
    # 3. Store in memory
    # 4. Return summary
    return [types.TextContent(type="text", text=json.dumps({
        "status": "loaded",
        "rows": len(df),
        "columns": list(df.columns),
    }))]
```

Pattern 2: Computation Tool

```
async def compute_tool(arguments: dict):
    # 1. Get data from memory
    df = get_dataframe(arguments.get("data_name"))

    # 2. Perform computation
    result = df[arguments["column"]].mean()

    # 3. Return result
    return [types.TextContent(type="text", text=str(result))]
```

Pattern 3: File I/O Tool

```
async def file_tool(arguments: dict):
    filename = arguments["filename"]
    content = arguments["content"]

    # 1. Write file
    with open(filename, "w") as f:
        f.write(content)

    # 2. Return confirmation
    return [types.TextContent(type="text", text=json.dumps({
        "status": "written",
        "file": filename,
    }))]
```

Summary

The MCP tools architecture provides:

1. **Modular Design**: Tools are independent, reusable components
2. **Protocol-Based Communication**: Standardized client-server interface
3. **Dynamic Discovery**: Agents discover tools at runtime
4. **Type Safety**: Schema validation ensures correct usage
5. **Easy Extension**: Add new tools with minimal code changes

Key Takeaways

- **Tools are the bridge** between LLM reasoning and real-world operations
- **Registry pattern** centralizes tool management
- **MCP protocol** provides a clean abstraction layer
- **LangGraph integration** enables multi-agent workflows
- **Error handling** at multiple levels ensures robustness

Next Steps

- Explore individual tool implementations in `tools/` directory
- Review agent prompts in `graph_v2.py` to understand tool usage

- Test tools via `main.py` demo requests
- Extend with custom tools following the patterns above